

Aula 00 - Primeiro fluxo imperativo de negocio

Contexto de negócio

Onboarding de cliente: registrar saldo inicial e validar uma primeira operacao sem violar regra de saldo.

Antes de discutir paradigmas, precisamos executar um fluxo real de conta: entrada de dados, processamento, decisao e saida.

Tensao operacional

Um exemplo apenas de `println` nao mostra como estado muda no programa.

Objetivo tecnico da entrega

- Fluxo imperativo em sequencia.
- Alteracao de estado em variavel.
- Decisao com `if`.

Recurso técnico desta aula

Fluxo executado nesta etapa:

- saldo inicial,
- deposito,
- tentativa de saque,
- saldo final.

Valor pratico entregue

- Reduz risco de saldo inconsistente em primeira transacao.
- A base fica pronta para evolucao controlada da proxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenario da aula.
- Executar a regra principal e observar o impacto no estado.
- Confirmar saida de sucesso, erro e borda antes de concluir a etapa.

Codigo em foco

```
// Classe temporaria de fluxo do programa.  
// Neste curso, ela existe para iniciar a execucao em Java.  
public class AppGestaoContas {  
    public static void main(String[] args) {  
        // Aula 00: fluxo imperativo completo com dado inicial, regra e saida.  
        String titular = "Ana";  
        double saldo = 1000.00;  
    }  
}
```

```
double deposito = 200.00;
double saque = 150.00;

System.out.println("Titular: " + titular);
System.out.println("Saldo inicial: " + saldo);

saldo = saldo + deposito;

if (saque <= saldo) {
    saldo = saldo - saque;
    System.out.println("Saque realizado: " + saque);
} else {
    System.out.println("Saque negado: saldo insuficiente.");
}

System.out.println("Saldo final: " + saldo);
}
```

Leitura tecnica orientada

Percorra o codigo com estes checkpoints de revisao:

- Regra de decisao: mapear condicoes que aprovam ou negam operacoes.
- Saida observavel: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Paradigma em foco: imperativo, procedural e algoritmo

Leitura de paradigma nesta etapa:

- Imperativo: o programa descreve passo a passo o que a máquina deve executar.
- Procedural: organizamos esse passo a passo em funções reutilizáveis.
- Algoritmo: sequência finita de entrada, processamento, decisão e saída para resolver um problema real.

Por que passar valores nas funções no procedural:

- A função não “conhece” a conta por contexto implícito.
- Tudo que a regra precisa deve chegar por parâmetro.
- O contrato da função fica explícito, testável e auditável.

Risco comum quando isso não fica claro:

- chamar função com parâmetro errado, em ordem errada ou incompleto;
- gerar resultado válido na sintaxe, mas incorreto no negócio.

Ponte para a próxima virada de paradigma:

- No procedural, o estado circula entre funções.
- Na POO, estado e comportamento passam a coexistir no objeto.

Perguntas de decisao

1. O que muda se o saque vier antes do deposito?
2. Quem garante a regra de saldo no programa?
3. Onde esta a regra de negocio neste exemplo?

Variacoes de cenario

1. Trocar saque para 2000.00.
2. Trocar deposito para 0.00.

Exercicios progressivos

1. Adicionar uma segunda tentativa de saque no mesmo fluxo.
2. Mostrar mensagem diferente para saque aprovado e negado.

Desafio de equipe

Adicionar variavel `tarifa` e descontar tarifa somente quando saque for aprovado.

Critérios de aceite dos exercicios

- Regra principal continua valida apos alteracoes.
- Cenario de erro foi testado e tratado sem quebrar execucao.
- Codigo permanece legivel e coerente com o nivel da aula.

Fechamento executivo

Esta aula mostra o pensamento imperativo completo. Na proxima, o foco sera reaproveitar regras com metodos.

Revisao rapida (5 minutos)

1. Regra central: qual condicao decide aprovacao ou bloqueio na aula?
2. Risco real: qual erro operacional essa implementacao reduz?
3. Evolucao: qual pequena extensao agregaria mais valor sem aumentar acoplamento?

Mini-missao de fechamento (5 min):

- Execute um cenario de borda no Tryit e justifique o resultado em 3 linhas.

Execucao no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteudo de `AppGestaoContas.java` no editor.
3. Clicar em Run, registrar saida base e depois testar variacoes e exercicios.
4. Manter uma aba separada para cada exercicio e comparar resultados.

Referencias W3Schools da aula

- https://www.w3schools.com/java/java_variables.asp

- https://www.w3schools.com/java/java_if_else.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>